Python for computational Problem Solving · Unit_03 · Unit 3: Functions, Functional Programming and Modules

DRAFT — UE26CS151A · Unit_03 · Unit 3: Functions, Functional Programming and Modules · Functional and object oriented programming

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · Functional and object oriented programming
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

Functional and object oriented programming

1 Functional and object oriented programming

Study focus: Functional and object oriented programming. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000026 · EV0000027

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

In today's class, we are going to be looking into the concept of functional programming.

Key details

- At the same time we will also un try to understand how.
- Python provides the different functions in order to work with the functional programming.
- Above all this, I really want to thank Professor Anis Kumar for providing us a beautiful examples in order to understand the concept of functional programming in an easy.
- I want you people to let me know what exactly these functions are trying to do.
- Fine, and there goes a parameter here, and then C.
- Define a list, an empty list in this case.
- What is this X?
- I will let.
- I'll say result.appen.
- Hope the functionality of an append with respect to a list.
- I'll say l of i.
- And after doing a computation, return the result for me.
- Having said this, let me create.
- This a list C B E S University.
- our very own Python.
- You can also write anything of your choice.
- Maybe more also, but I'll stop at this particular point.
- F1 and I will pass A as an argument here.
- I'll capture the result and then print it.
- I hope by this time you should have known what I'm trying to do here.

- And if you can see the result is fourteen, six and sixteen.
- What exactly is this?
- If you can see count this three, four, five, six, seven, eight, nine, ten, 10 plus 4, 14.
- Let me create a function.
- In this case, my function will be F2.
- There's got to be a parameter here also, which is x.
- I will just copy the same thing.
- But what I'll do is I'll do some changes.
- Hope this is okay with you all.
- And let me comment this so that you will not have any confusions here.
- Yeah, here we go.
- STR dot upper.
- STR dot upper.
- I'll have to change this because this is function two.
- And let me execute.
- When I execute this program, if you can see that all the elements of a list have been converted to its upper case.
- The observation goes like this.
- The observation is some initialization has been done.
- I have initialized certain things.
- And we are traversing through all the elements of an I trouble in this case.
- Is this fine?
- We have gone through all the elements of a list.
- List is an I tribble for you.

Example or application from the lecture

- Before I start with actual functional programming, let us try to write very simple programs and then understand what those programs are doing.
- So let me do one more simple example for you.
- Now, in both the examples, in the example one, where we had the function f1, and in this case this is my function f2, there are few things that we need to observe.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000026 · EV0000027 · EV0000028 · +1 source units in pack.json

Core explanation · 05:00–09:59

We are trying to apply some function to this.

Key details

- We have applied some function.
- In this case, the result is collected again in terms of a list.
- And then followed by the list has been returned.
- We did some initialization.
- Some initialization have been done.

- I am walking through every element of an I trouble.
- I take every element from an I tribble and apply some operation.
- And result is collected again in terms of an I triple itself, but in this case it is a list and I will return the list.
- The same thing we are trying to do here also.
- Okay, so now, so now what we are going to be doing next.
- In this case, I will rewrite the same set of programs in a different manner.
- Just look at how I will write the program.
- I will use the same list.
- Fine, I'll copy paste it here.
- Let me remove this.
- Hope this is not confusing to you.
- Just give me a time.
- I will remove this.
- And I will comment it till here.
- I do not have anything else here.
- This is my list.
- I'll say A is equal to map.
- Map of what?
- Map of Len, comma, A.
- Hope this is fine.
- Let me see print A.
- It's just fine.
- What am I trying to do?
- I'm making use of a function.
- In this case, map is a built-in function.
- Please let me remind you, map is a built-in function.
- Map is a built-in function which takes a callable.
- Along with that, it also takes an eye trouble.
- And hope I am not confusing you.
- I have a map.
- For this map.
- What did I do?
- And then I am calling a particular function.
- What is this function?
- This function is a callable function.
- I can call it as and when it is required.
- Hope there are no more confusions.
- As I mentioned, map takes two arguments, a callable and an iterable.
- What am I trying to do?
- What am I trying to do?
- Before me explaining, let me get the output first and then I will try to explain it.
- And if you can see, you're getting some map objects.
- You're getting a map object.

- What I will do is I will just change something like this, I'll say list of this.
- Is this fine?

Example or application from the lecture

- In this example, at the same time in the previous example also we have done this.
- You can see the previous example, aren't we doing the same thing here?
- For this particular map, I have given an I trouble fine and I have called a particular function len in this case.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000030 · EV0000031 · EV0000032 · +1 source units in pack.json

Core explanation · 10:00–14:59

That is what I'm trying to do.

Key details

- The reason is map is lazy.
- Map is lazy.
- Let me write map is lazy.
- Because of which what did I do?
- I said list of this.
- If you could not understand that particular statement, don't worry, you have an item, element, on this particular element, okay, on this particular element, what am I doing?
- I will apply this particular function.
- That's it.
- Nothing else we are trying to do.
- And one thing you should always remember when you work with your map function, when you work with your map function, the number of input elements will be equal to the number of output elements.
- That is the very important thing that you should always remember.
- Number of input elements will always be equal to the number of output elements.
- The one change is instead of len, I will say string dot upper.
- And that's it.
- Is this fine?
- You're saying take every element in an I tribble and apply a callable on it.
- That's what you are trying to do.
- Hope our pro problem is saved.
- Program is saved, and if you can see they've been converted into capital letters.
- Work out on the similar grounds so that you can understand the things in an easier manner.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000034 · EV0000035 · EV0000036 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Functional and object oriented programming transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/007_Functional_and_object_oriented_programming__8545ef40cb22fe08.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Functional and object oriented programming transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/007_Functional_and_object_oriented_programming__8545ef40cb22fe08.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Functional and object oriented programming transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/007_Functional_and_object_oriented_programming__8545ef40cb22fe08.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9986; only the JSON segmentation supplies the displayed timestamps.
Functional and object oriented programming transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/007_Functional_and_object_oriented_programming__8545ef40cb22fe08.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.

2. 4 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00010	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0053. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00011	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0053. Sidecar variants compare...
SRC00012	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0053. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00013	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0053. Sidecar variants compare...

C Revision and retrieval practice